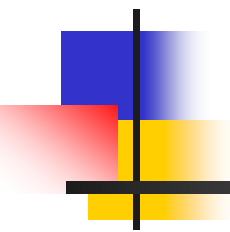


3D Computer Graphics



Rasterization

Dan Biediger



UNIVERSITY *of* HOUSTON

DEPARTMENT OF COMPUTER SCIENCE

Polygon Meshes

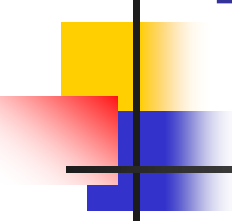


Life of Pi (2012)

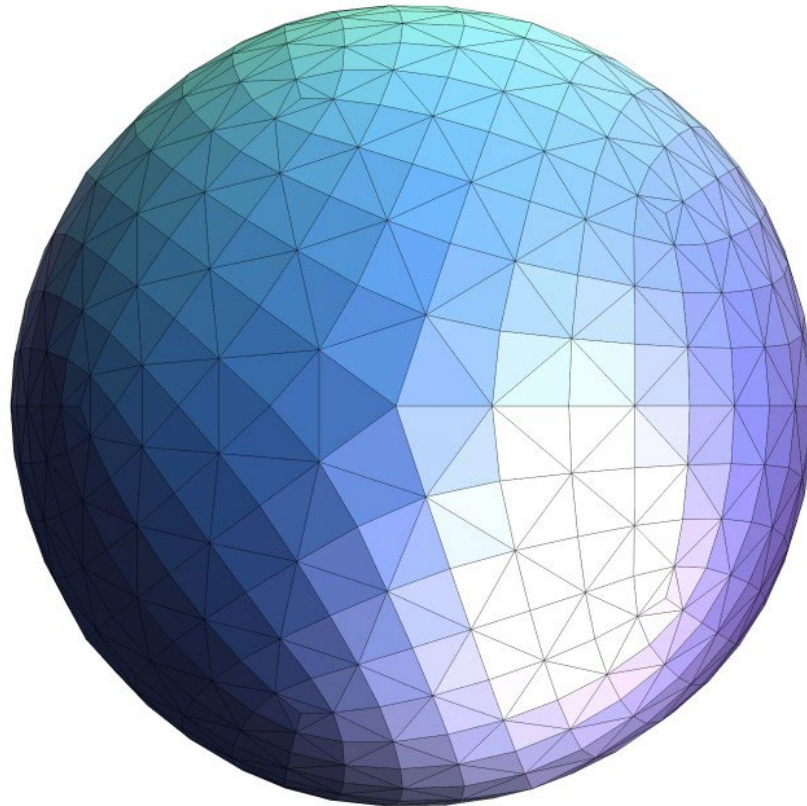


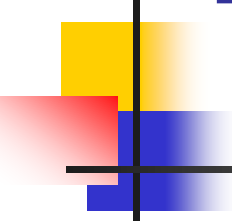
UNIVERSITY *of* **HOUSTON**

DEPARTMENT OF COMPUTER SCIENCE

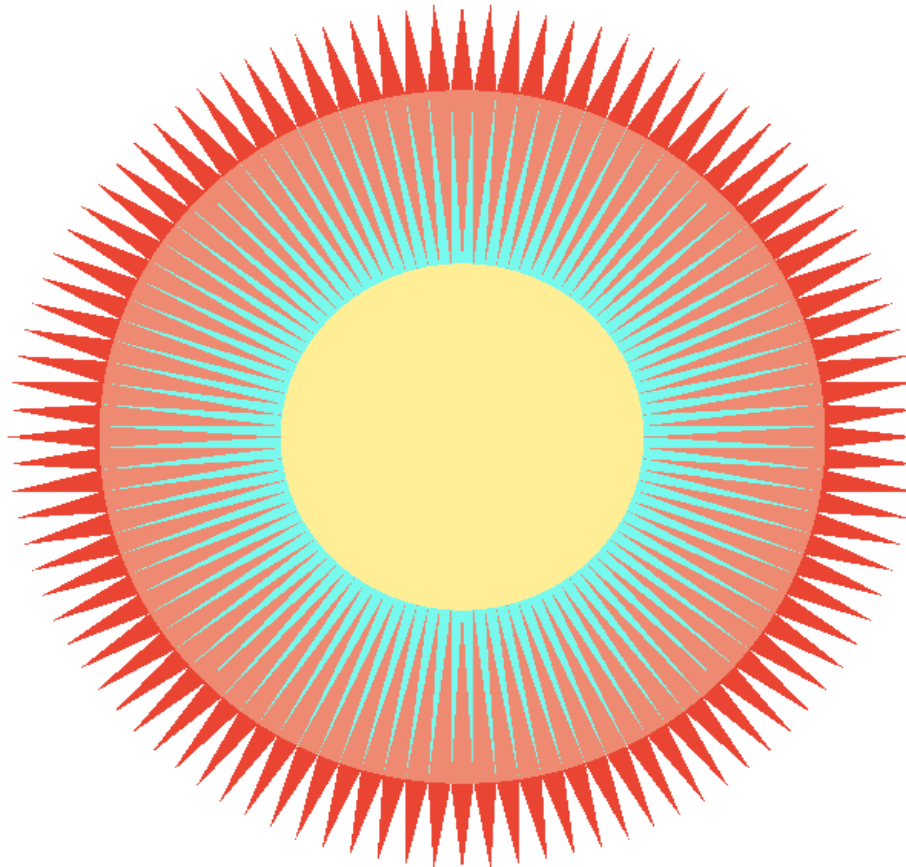


Triangle Meshes





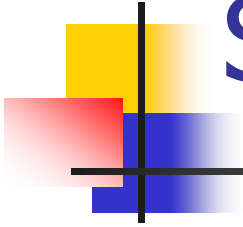
Triangle Meshes



UNIVERSITY *of* **HOUSTON**

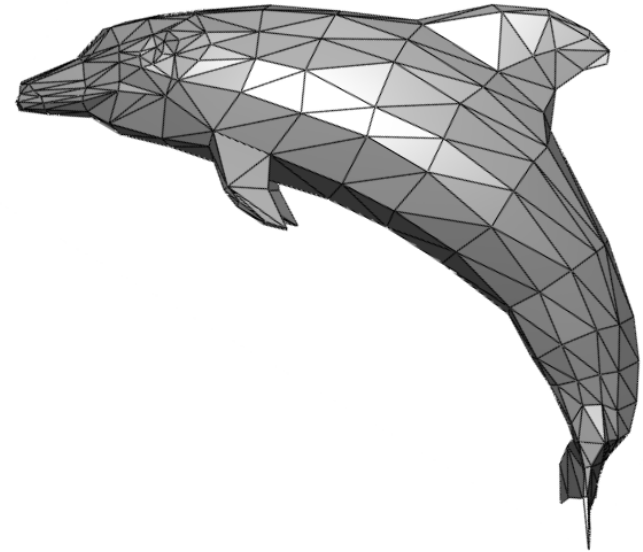
DEPARTMENT OF COMPUTER SCIENCE

Triangles - Fundamental Shape Primitives

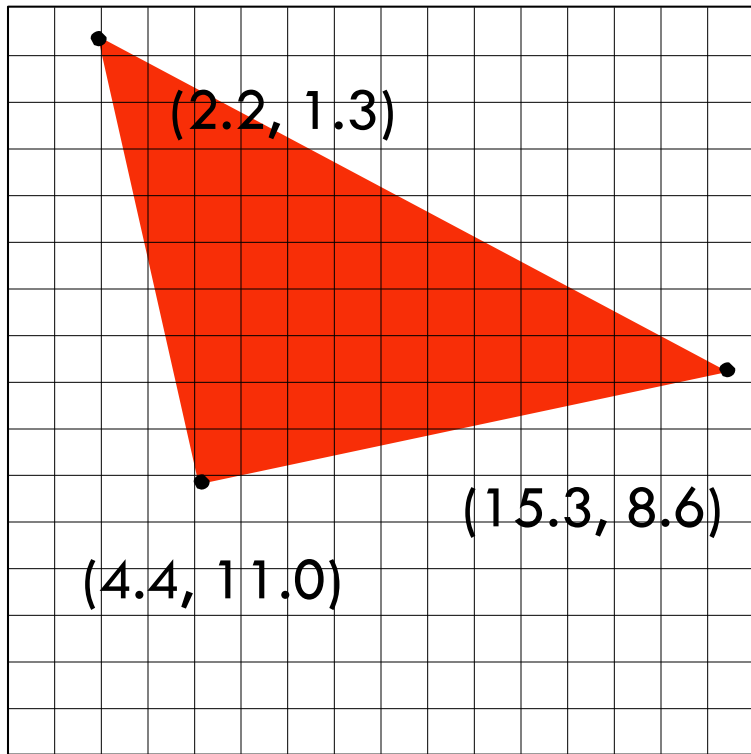


Why triangles?

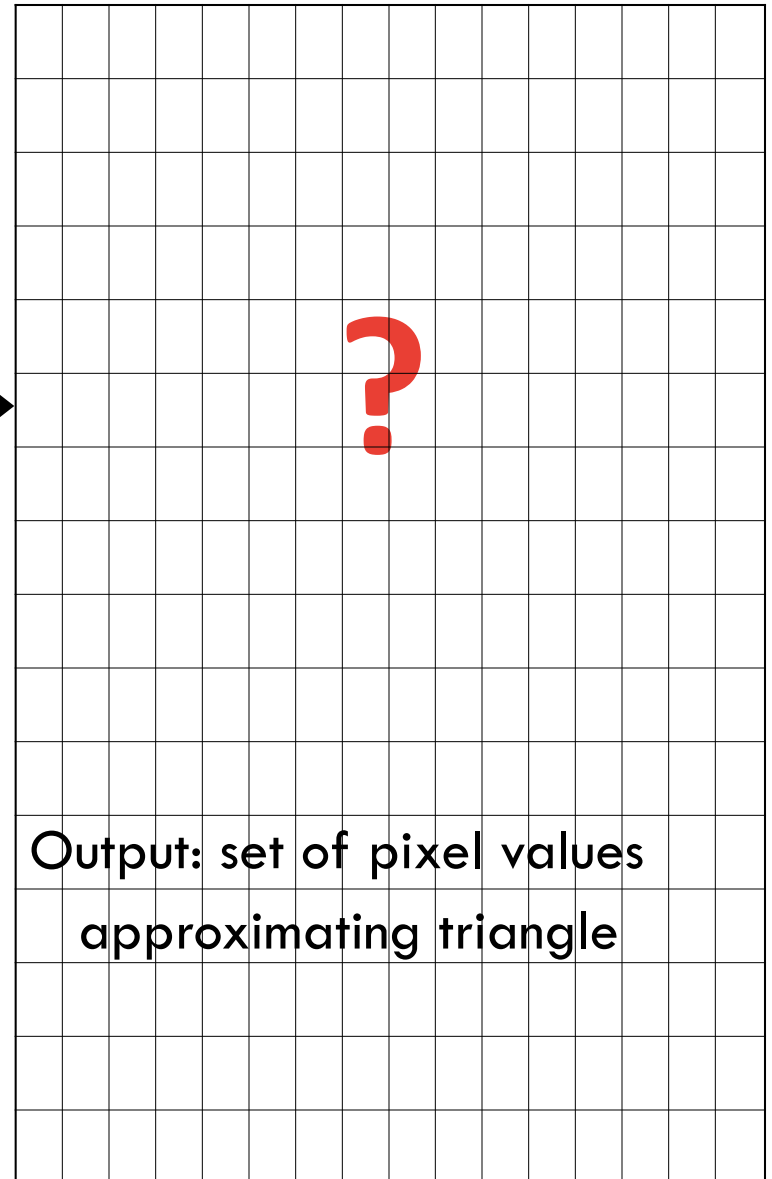
- Most basic polygon
 - Break up other polygons
- Unique properties
 - Guaranteed to be planar
 - Well-defined interior
 - Well-defined method for interpolating values at vertices over triangle (barycentric interpolation)

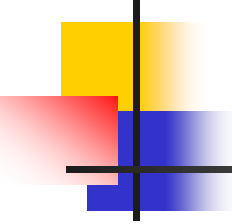


What Pixel Values Approximate a Triangle?



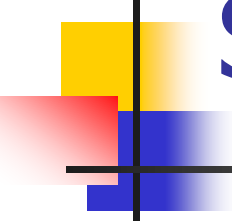
Input: position of triangle
vertices projected on screen





A Simple Approach: Sampling





Sampling a Function

Evaluating a function at a point is sampling. We can **discretize** a function by sampling.

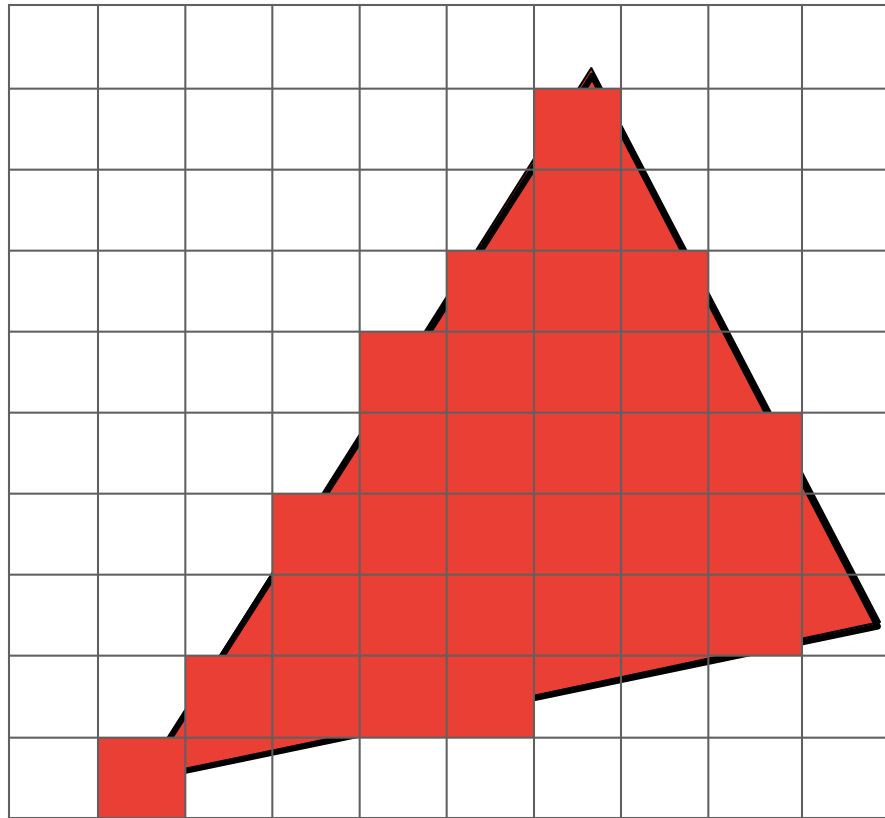
```
for (int x = 0; x < xmax; ++x)
    output[x] = f(x);
```

Sampling is a core idea in graphics.

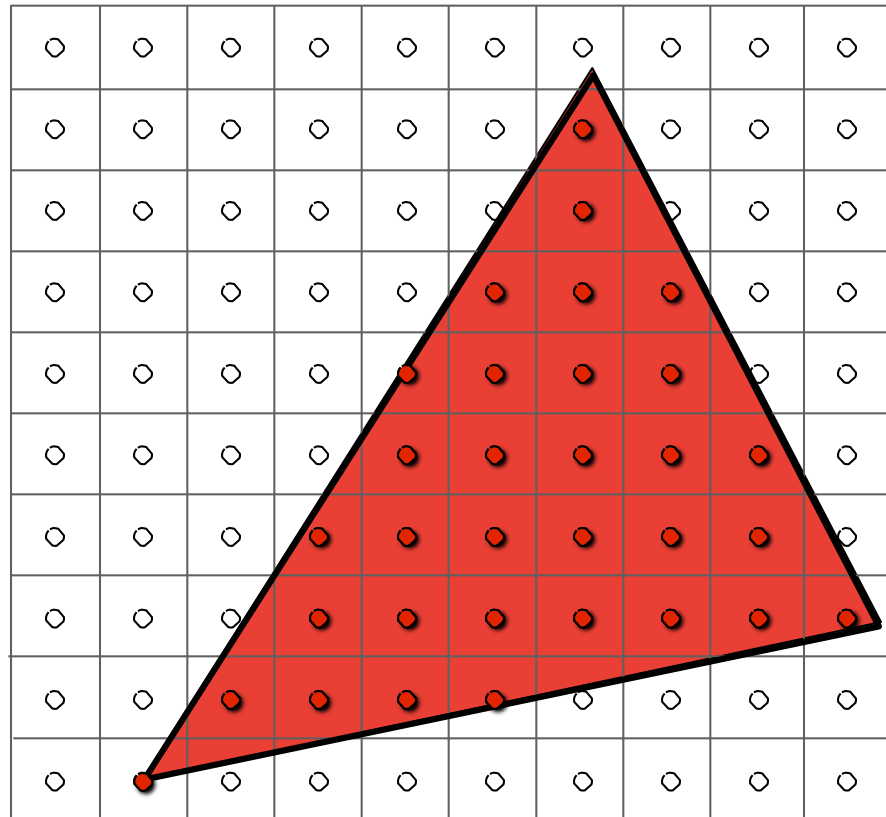
We sample time (1D), area (2D), direction (2D), volume (3D) ...



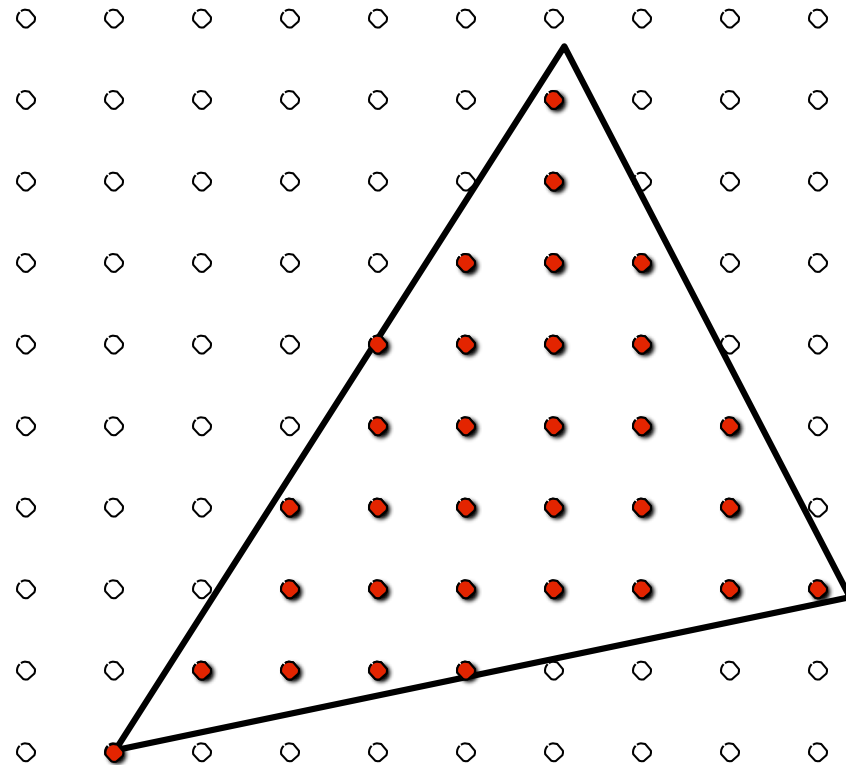
Rasterization As 2D Sampling

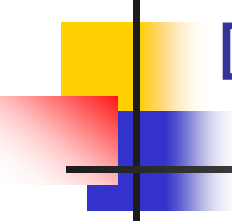


Sample If Each Pixel Center Is Inside Triangle



Sample If Each Pixel Center Is Inside Triangle

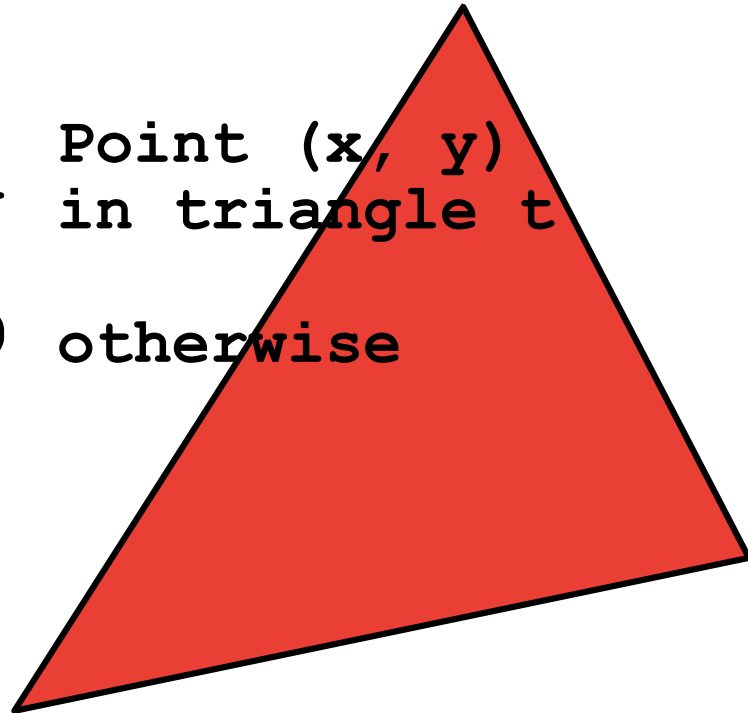


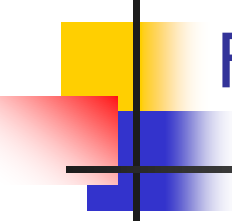


Define Binary Function: `inside(tri, x, y)`

`x, y`: not necessarily integers

$$\text{inside}(t, x, y) = \begin{cases} 1 & \text{Point } (x, y) \\ & \text{in triangle } t \\ 0 & \text{otherwise} \end{cases}$$



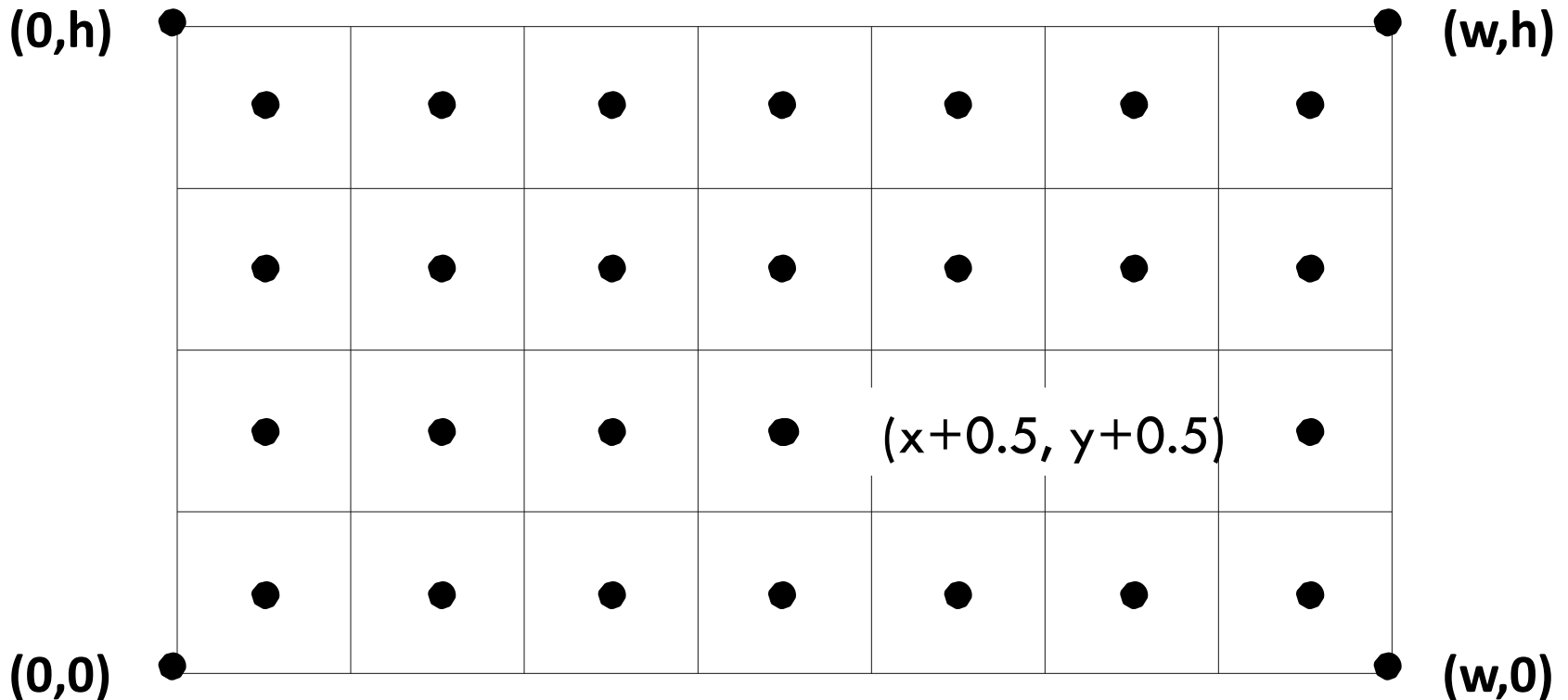


Rasterization = Sampling A 2D Indicator Function

```
for (int x = 0; x < xmax; ++x)
    for (int y = 0; y < ymax; ++y)
        image[x][y] = inside(tri,
                               x + 0.5,
                               y + 0.5);
```

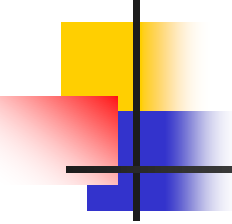


Recall: Sample Locations

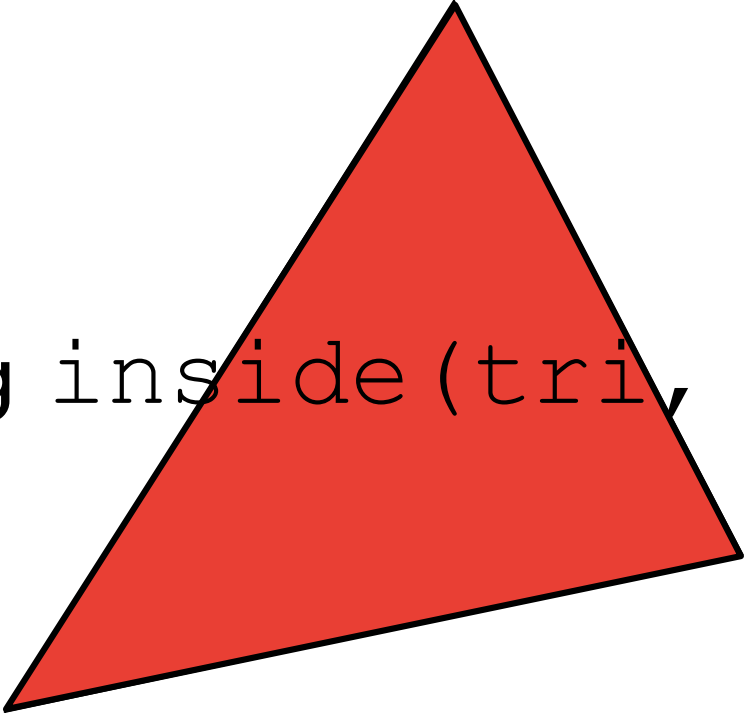


Sample location for pixel (x, y)

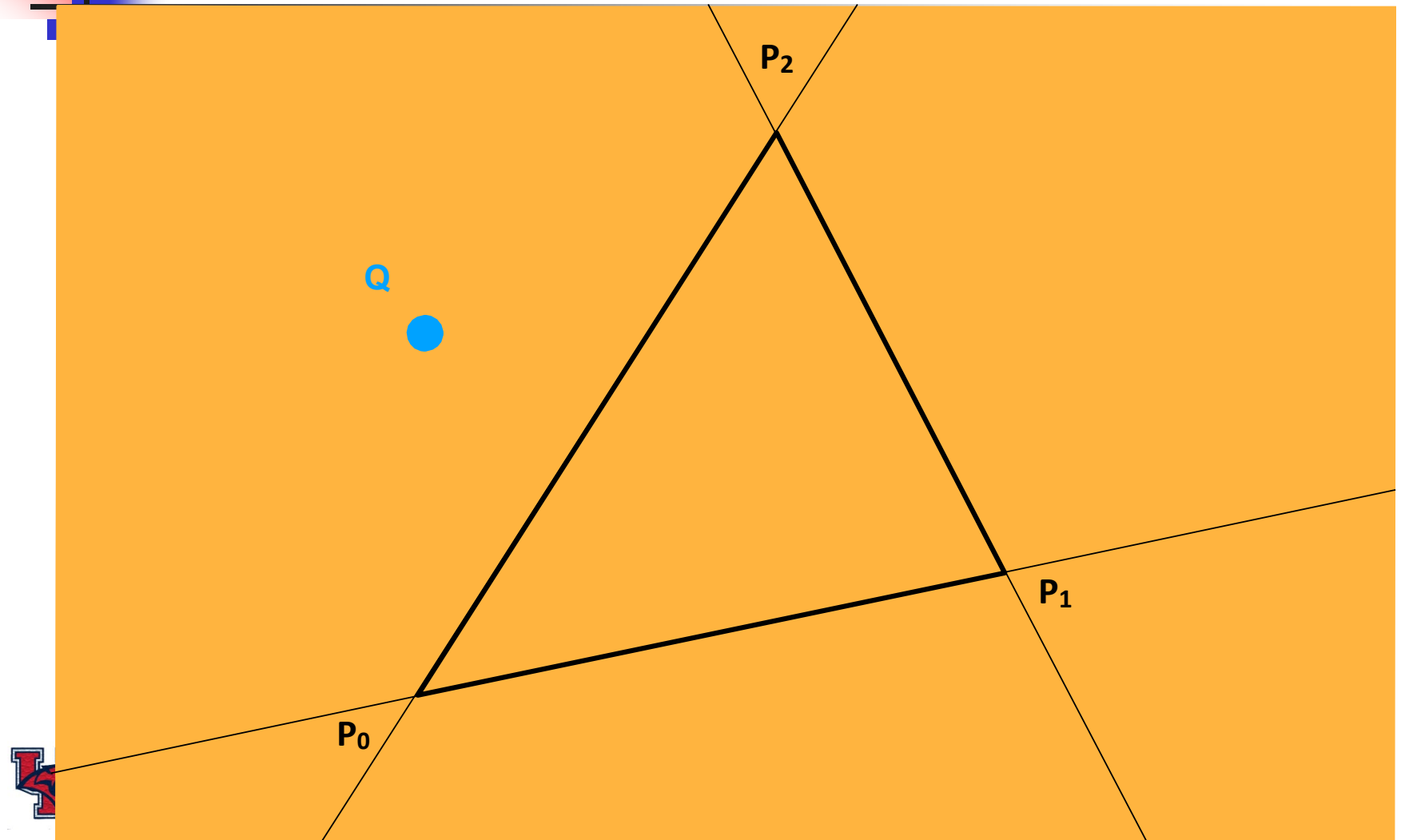


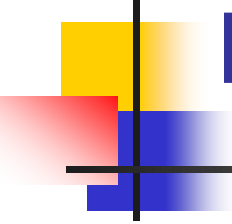


Evaluating `inside(tri, x, y)`



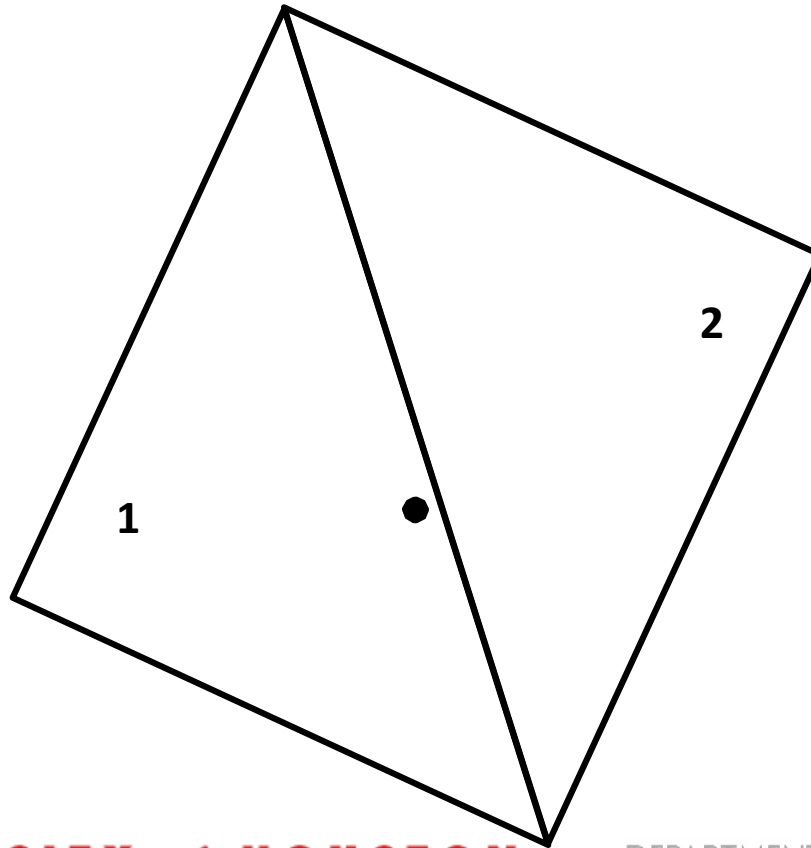
Inside? Three Cross Products!





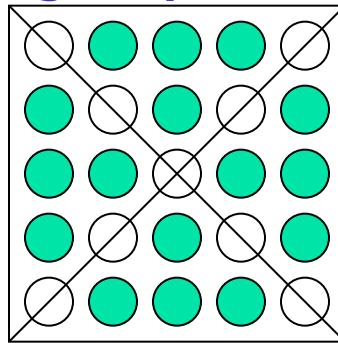
Edge Cases (Literally)

Is this sample point covered by triangle 1, triangle 2, or both?

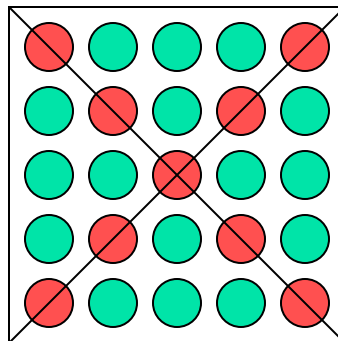


Handling Edge Pixels

Don't use edges ($e==0$) – missing pixels



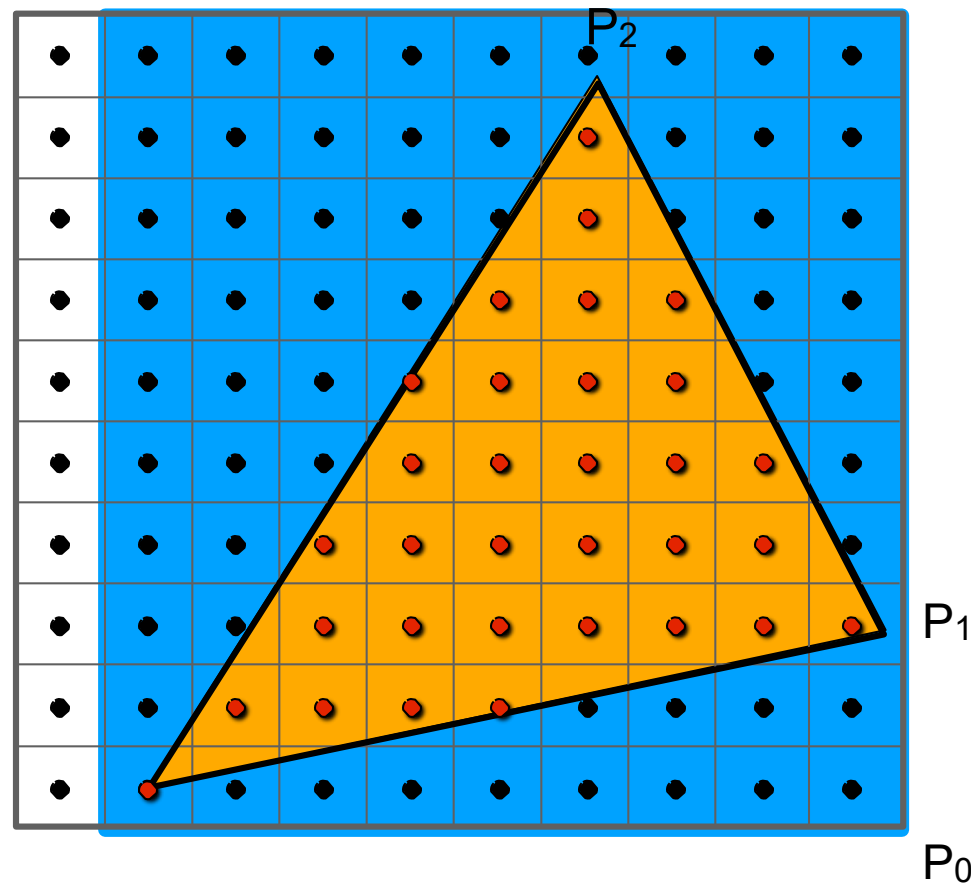
Always use edges ($e==0$) – waste & flicker



Need to include edge pixels on left or right edges (also determined by sort) to avoid pinholes between tris.



Checking All Pixels on the Screen?



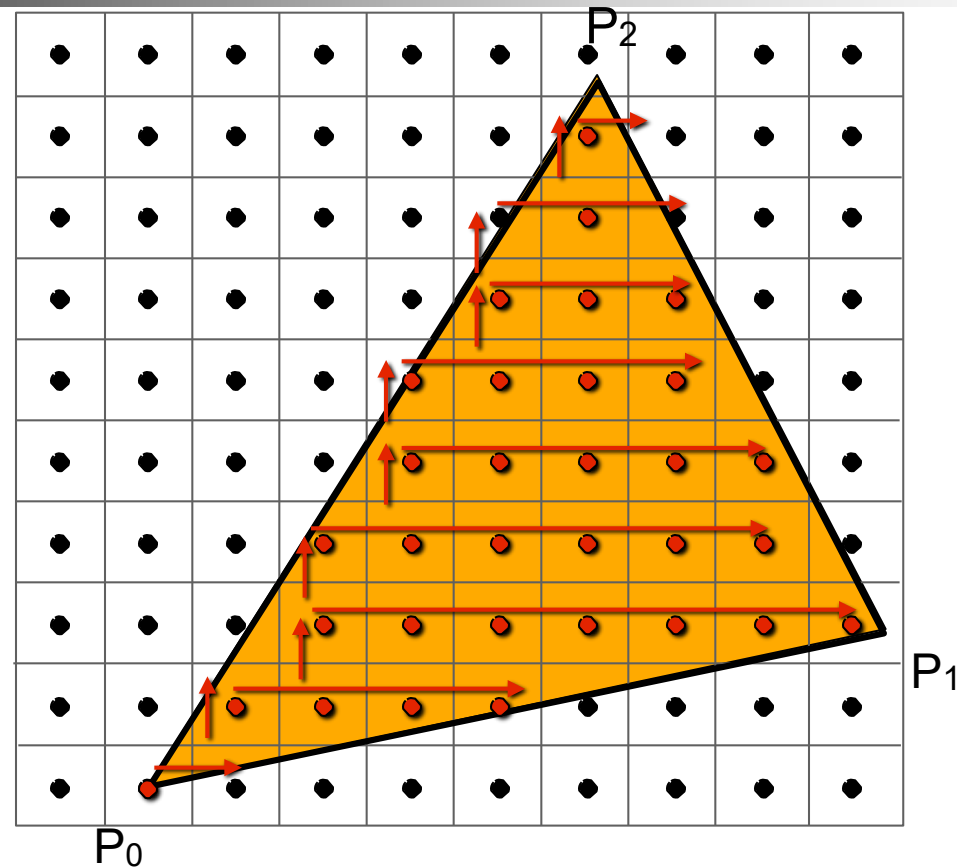
Use a **Bounding Box!**



UNIVERSITY *of* **HOUSTON**

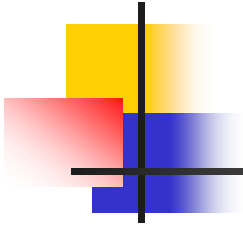
DEPARTMENT OF COMPUTER SCIENCE

Incremental Triangle Traversal (Faster?)



suitable for thin and rotated triangles

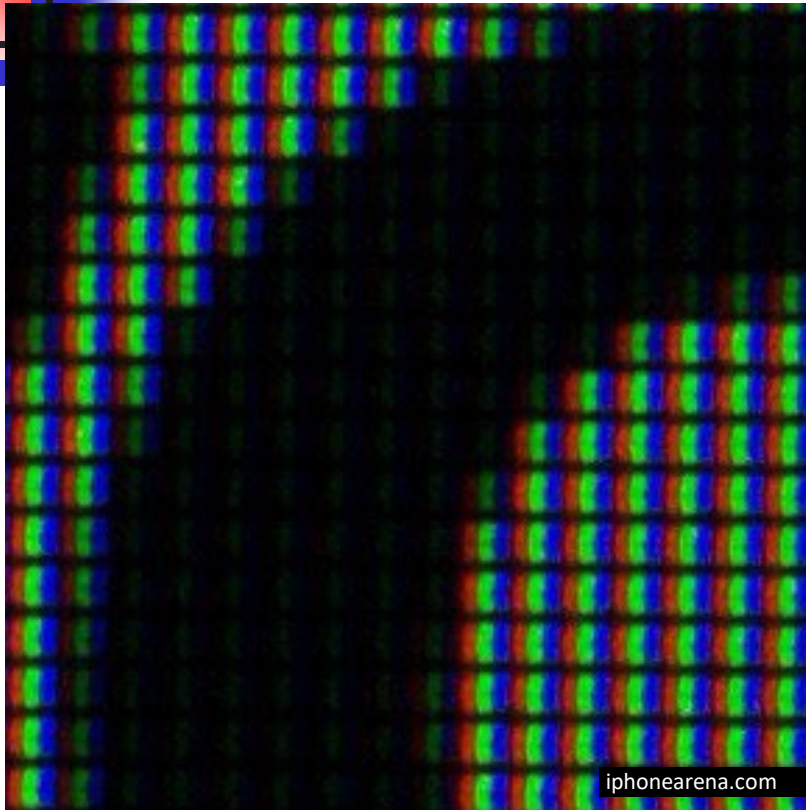




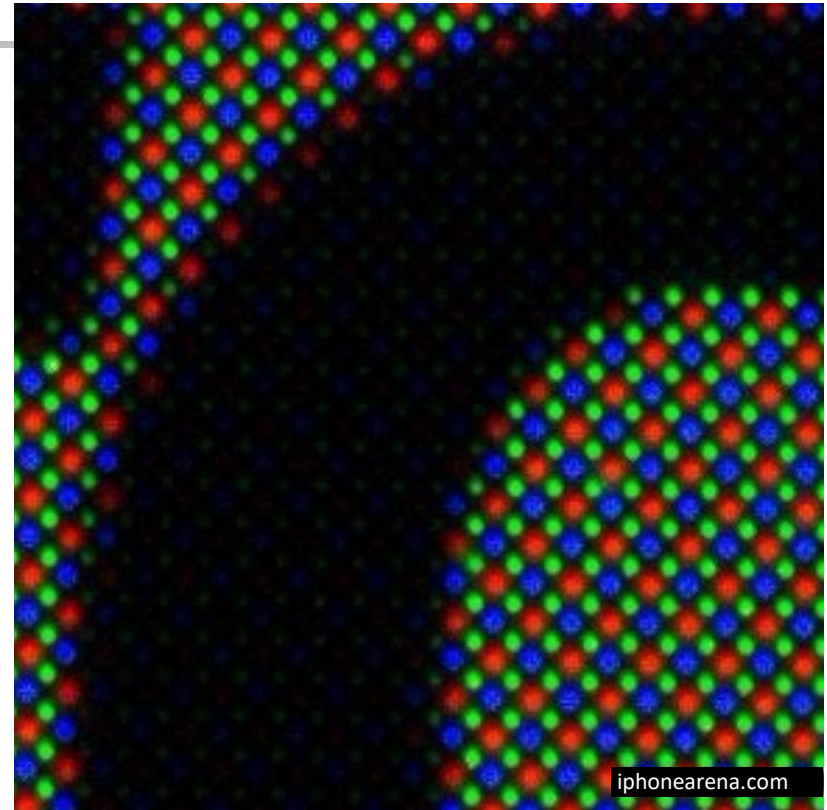
Rasterization on Real Displays



Real LCD Screen Pixels (Closeup)



iPhone 6S



Galaxy S5

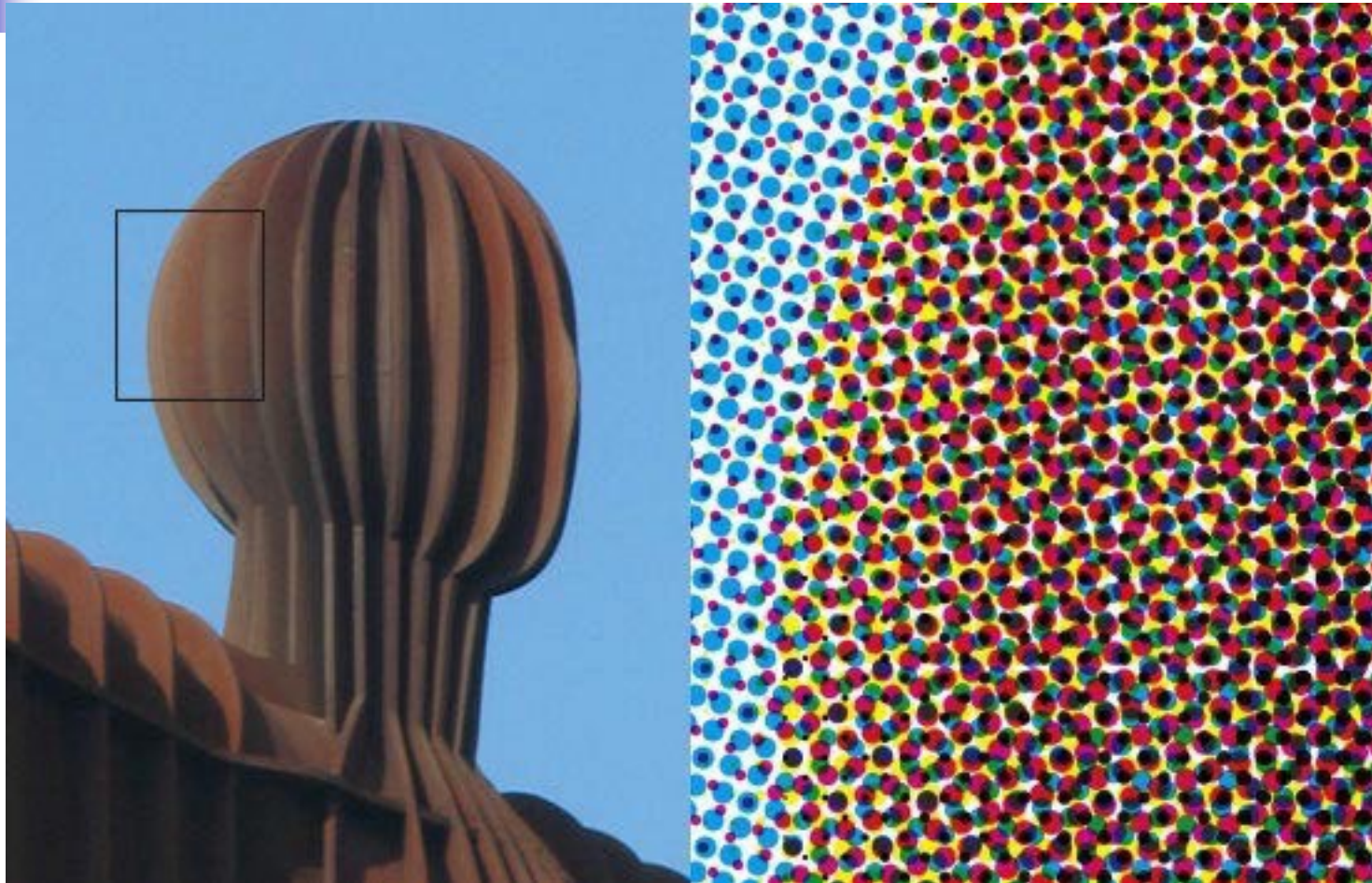
Notice R,G,B pixel geometry! But in this class, we will assume a colored square full-color pixel.



UNIVERSITY *of* **HOUSTON**

DEPARTMENT OF COMPUTER SCIENCE

Aside: What About Other Display Methods?

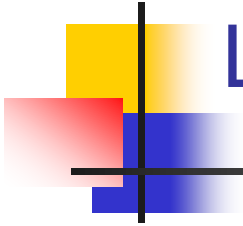


UNIVERSITY *of* **HOUSTON**

DEPARTMENT OF COMPUTER SCIENCE

Color print: observe half-tone pattern

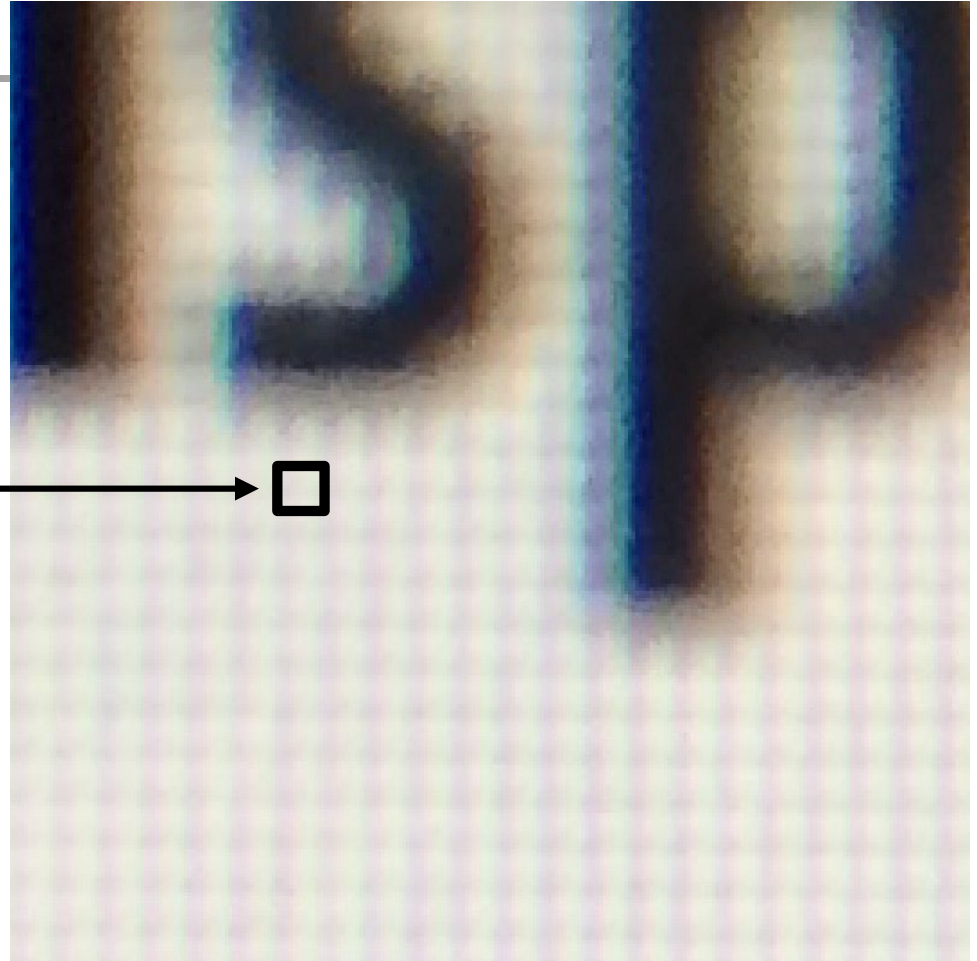
Assume Display Pixels Emit Square of Light



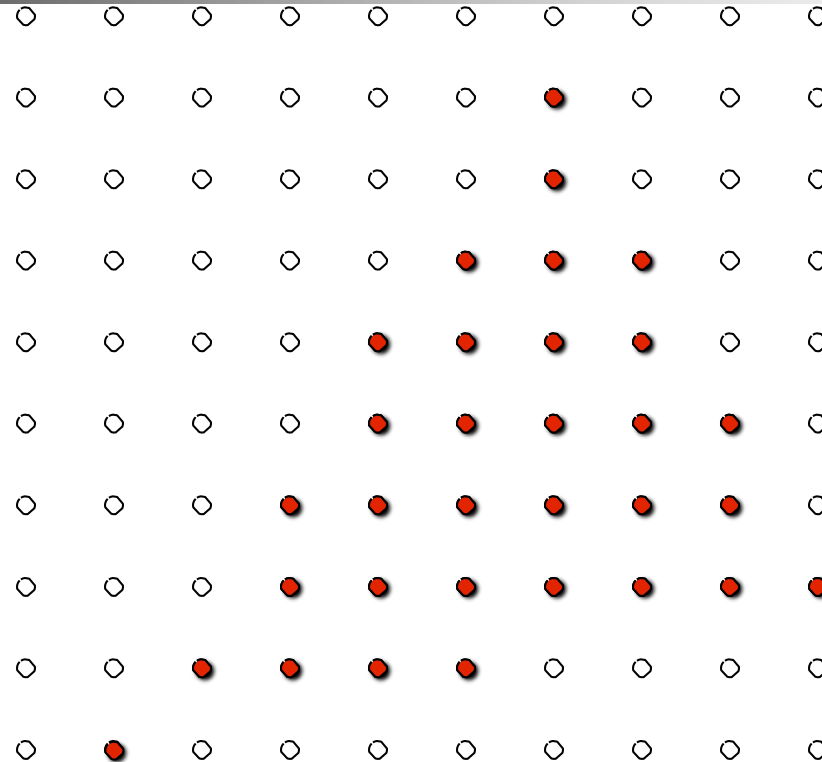
LCD pixel on laptop



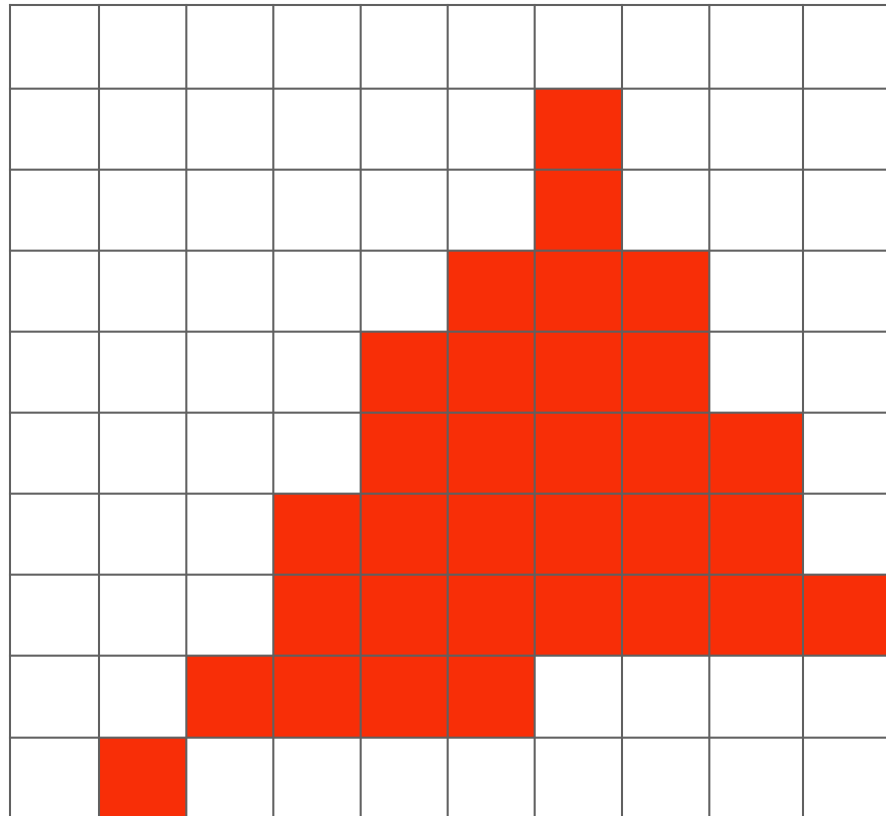
* LCD pixels do not actually emit light in a square of uniform color, but this approximation suffices for our current discussion



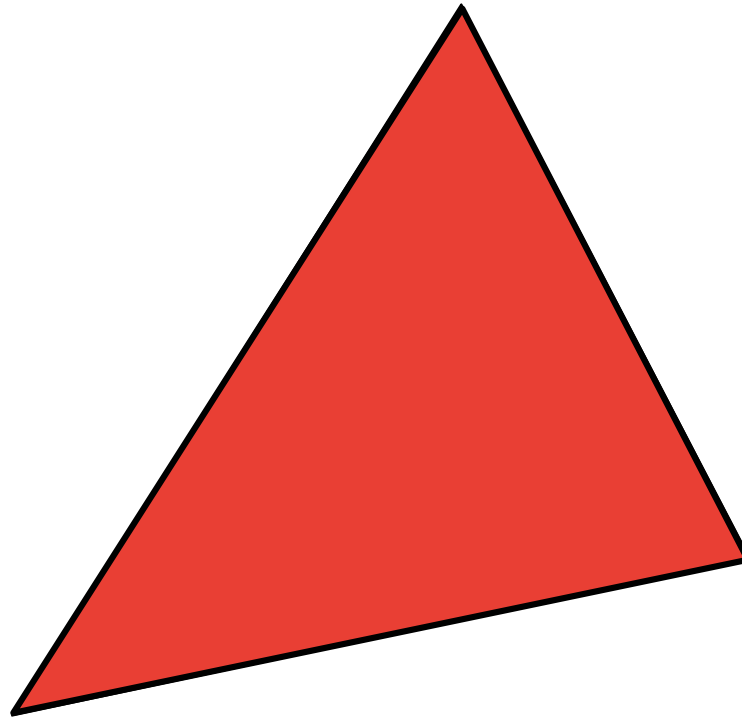
So, If We Send the Display the Sampled Signal



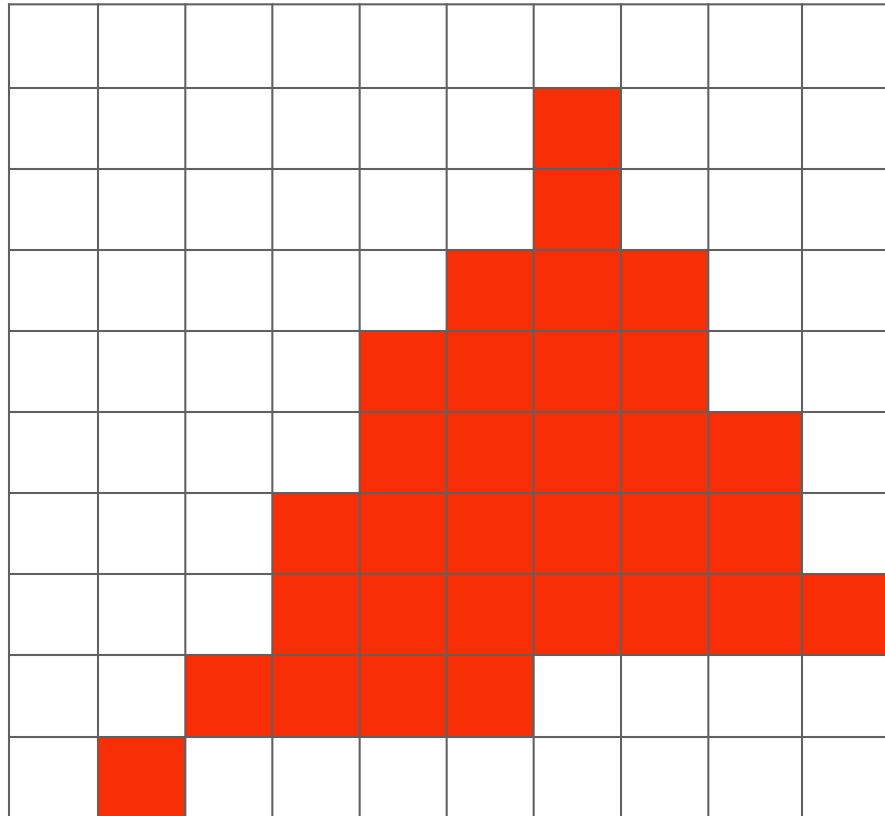
The Display Physically Emits This Signal



Compare: The Continuous Triangle Function



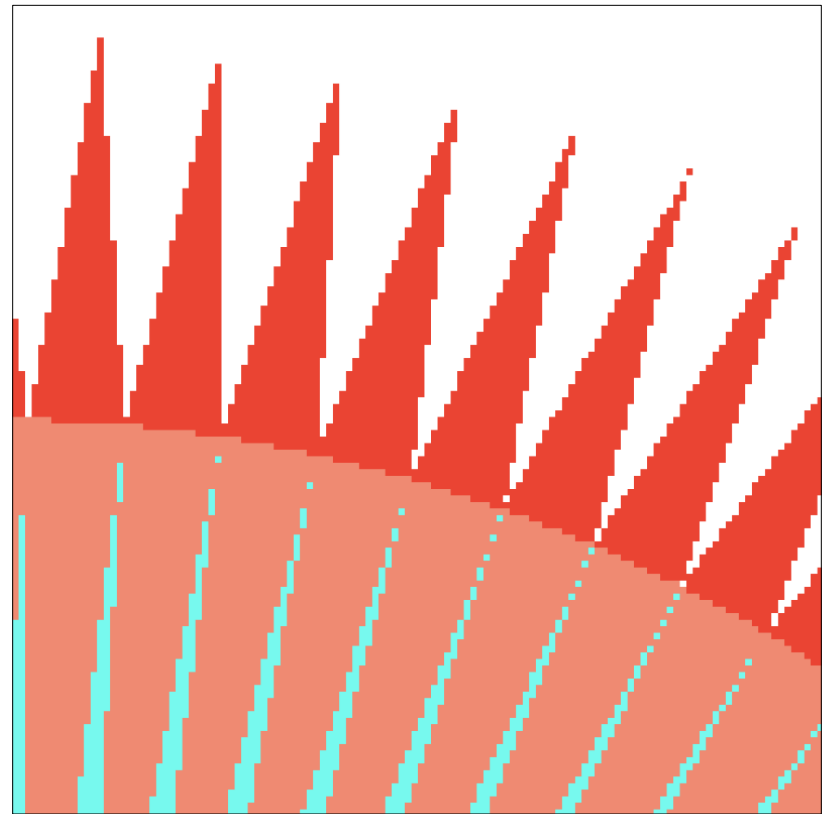
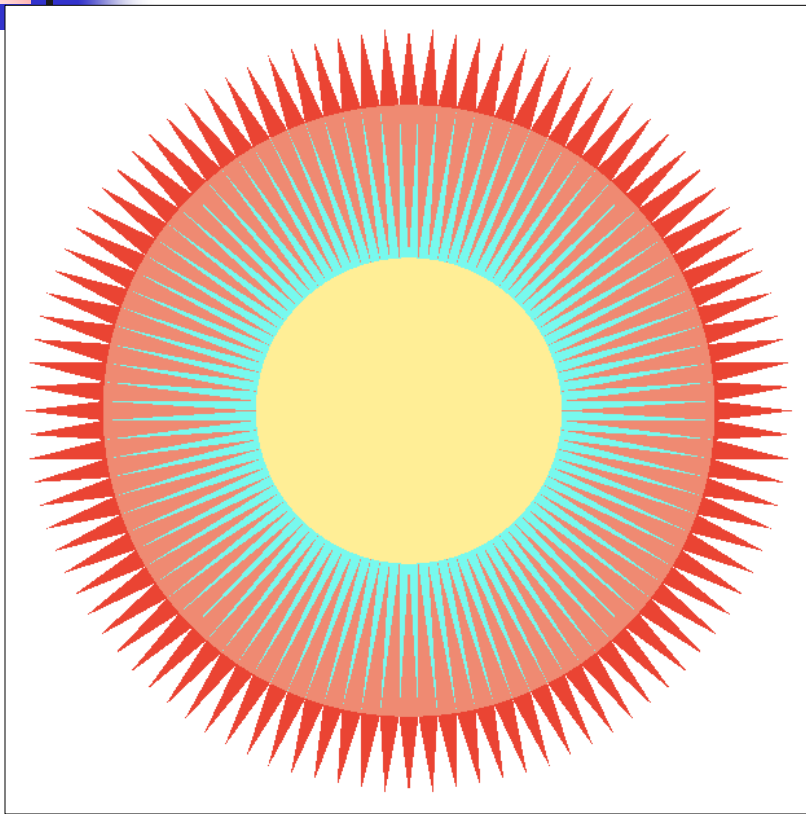
What's Wrong With This Picture?



Jaggies!



Aliasing (Jaggies)

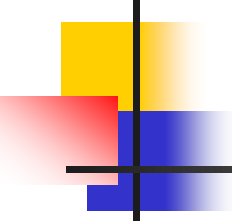


Is this the best we can do?



UNIVERSITY *of* **HOUSTON**

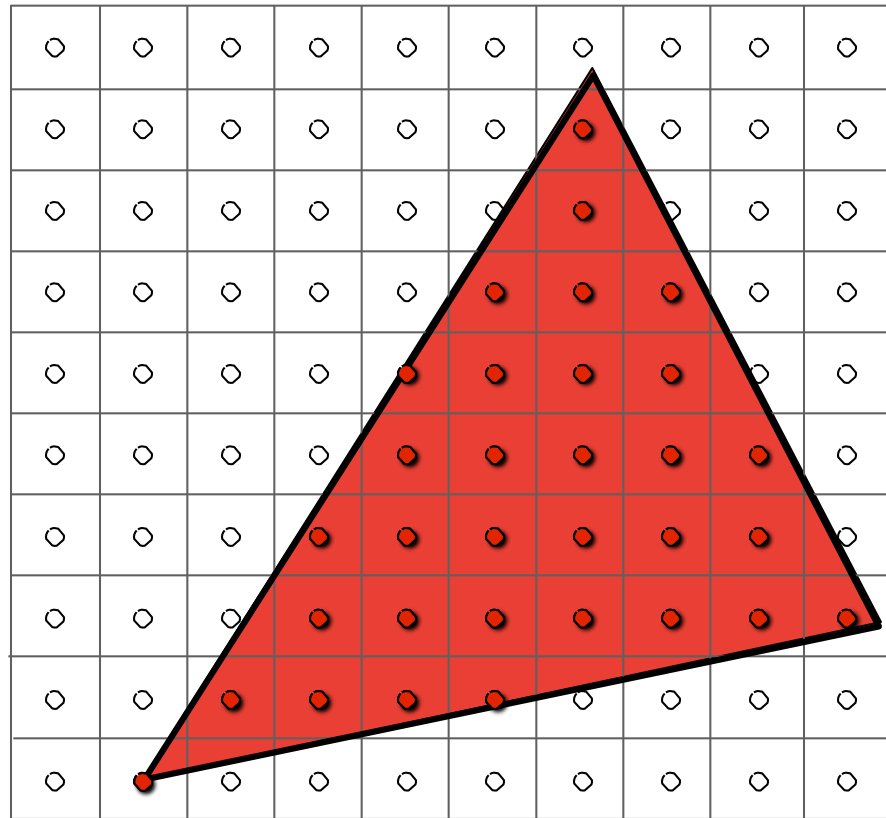
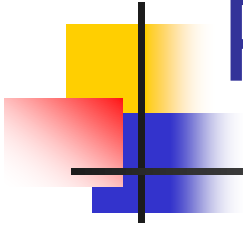
DEPARTMENT OF COMPUTER SCIENCE



Sampling is Ubiquitous in Computer Graphics



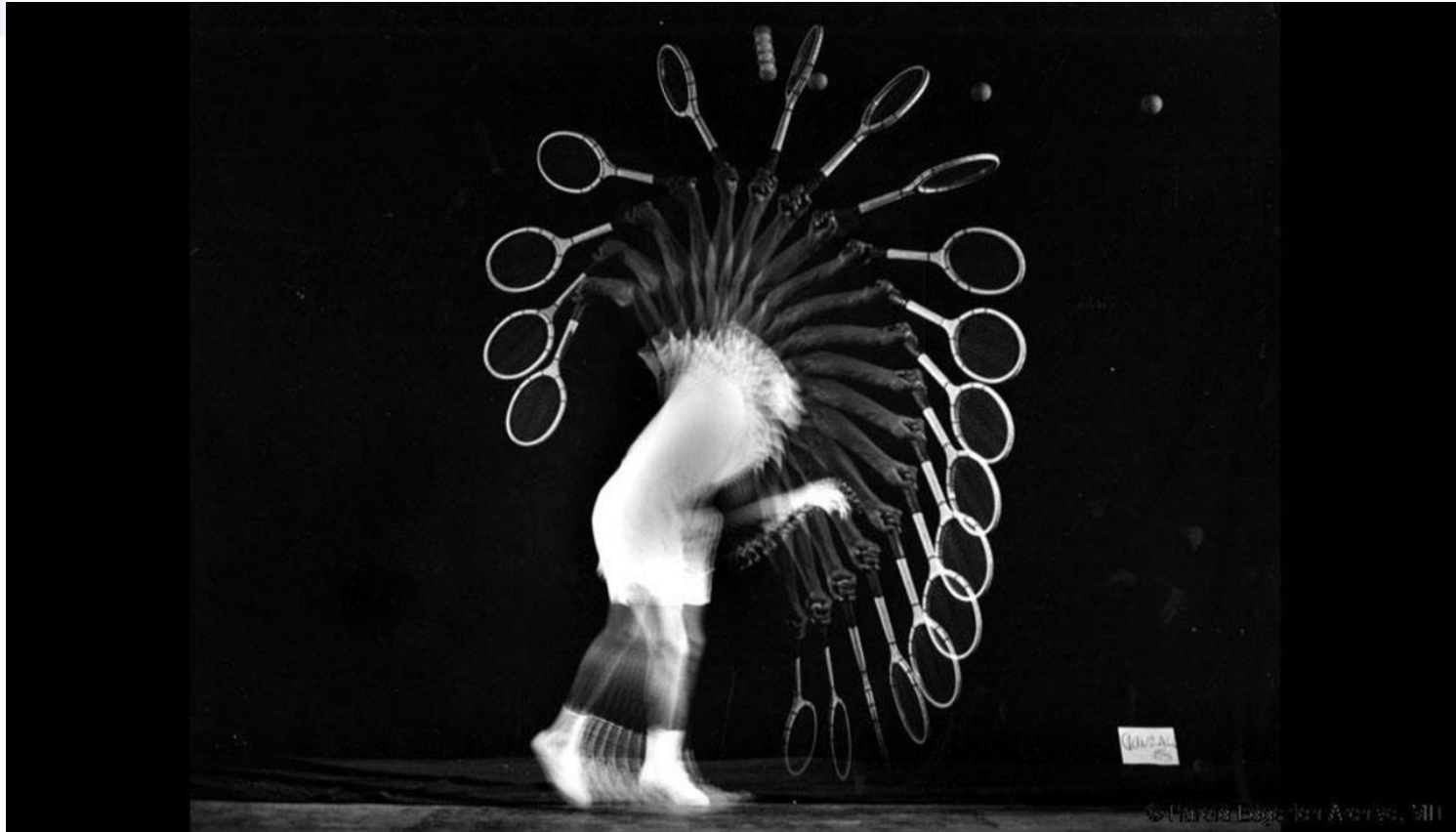
Rasterization = Sample 2D Positions



Photograph = Sample Image Sensor Plane



Video = Sample Time

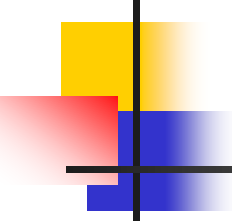


Harold Edgerton Archive, MIT



UNIVERSITY of HOUSTON

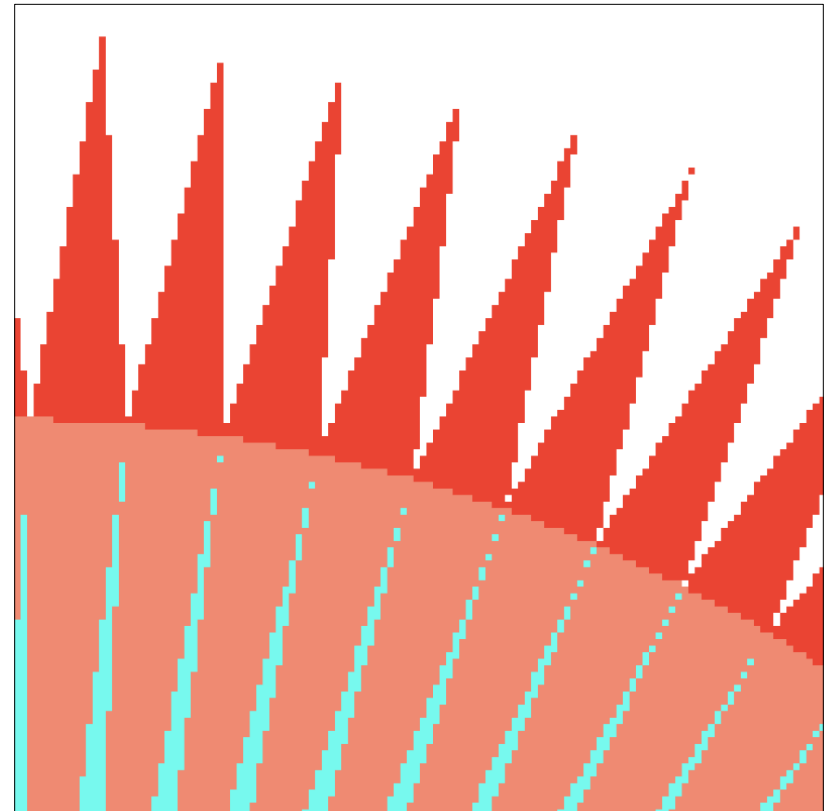
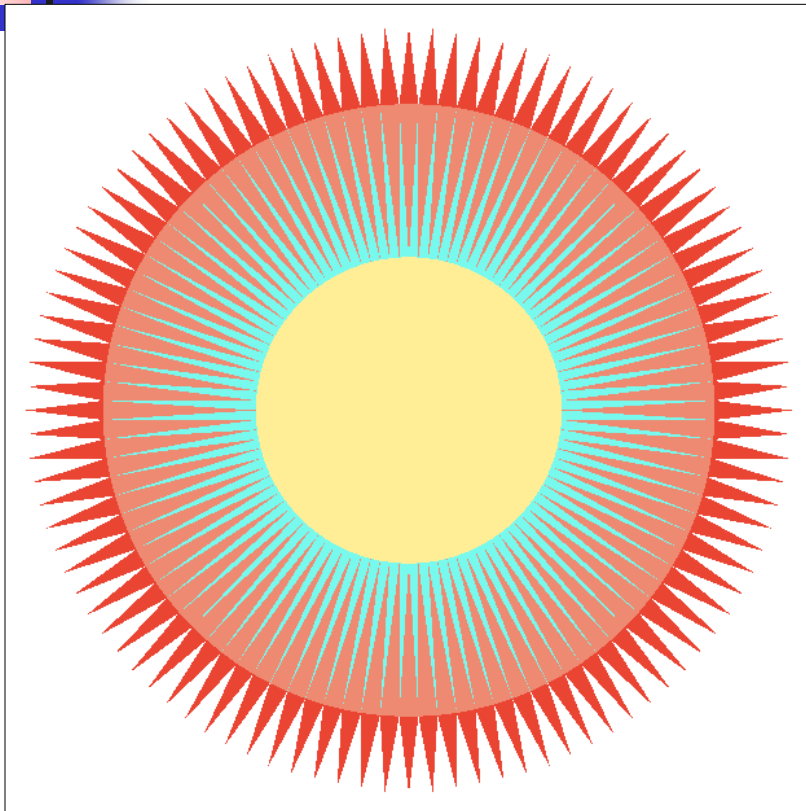
DEPARTMENT OF COMPUTER SCIENCE



Sampling **Artifacts** (Errors / Mistakes / Inaccuracies) in Computer Graphics



Jaggies (Staircase Pattern)



This is also an example of “aliasing” – a sampling error



UNIVERSITY *of* **HOUSTON**

DEPARTMENT OF COMPUTER SCIENCE

Moiré Patterns in Imaging



Skip odd rows and columns



UNIVERSITY *of* **HOUSTON**

DEPARTMENT OF COMPUTER SCIENCE

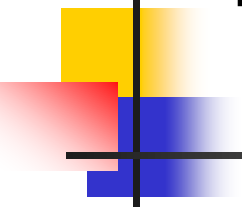
Wagon Wheel Illusion (False Motion)



https://www.youtube.com/watch?v=QOwzkND_ooU



Sampling Artifacts in Computer Graphics



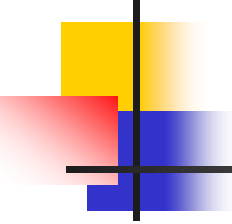
Artifacts due to sampling - “Aliasing”

- Jaggies – sampling in space
- Moire – undersampling images
- Wagon wheel effect – sampling in time
- [Many more] ...

Behind the Aliasing Artifacts

- Signals are **changing too fast** (high frequency), but **sampled too slowly**

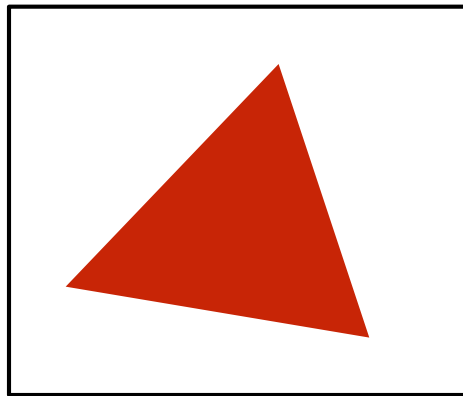




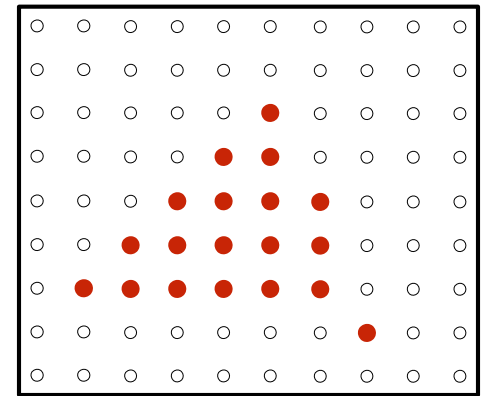
Antialiasing Idea: Blurring (Pre-Filtering) Before Sampling



Rasterization: Point Sampling in Space



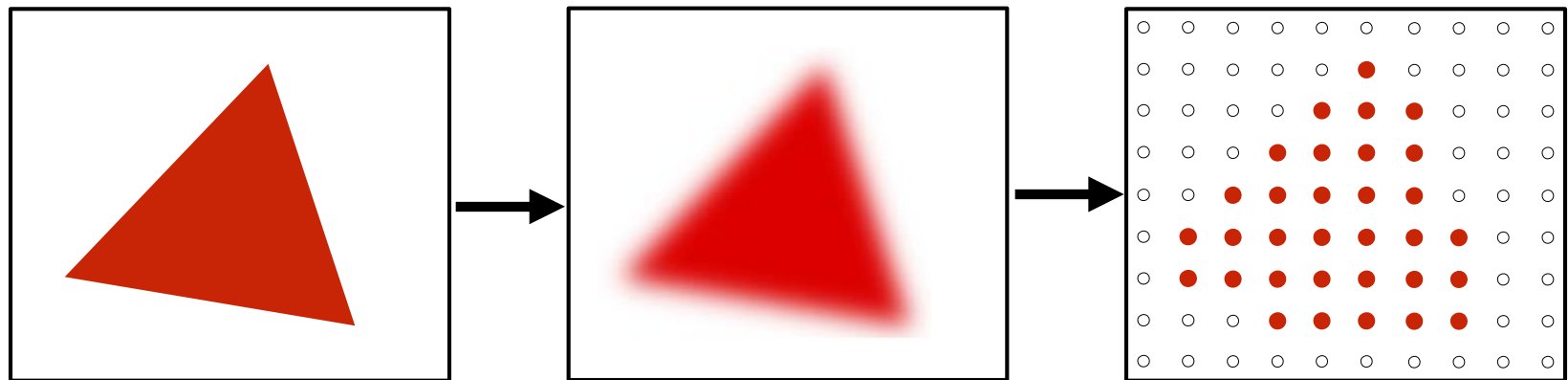
Sample



Note jaggies in rasterized triangle where pixel values are **pure red or white**



Rasterization: Antialiased Sampling



Pre-Filter

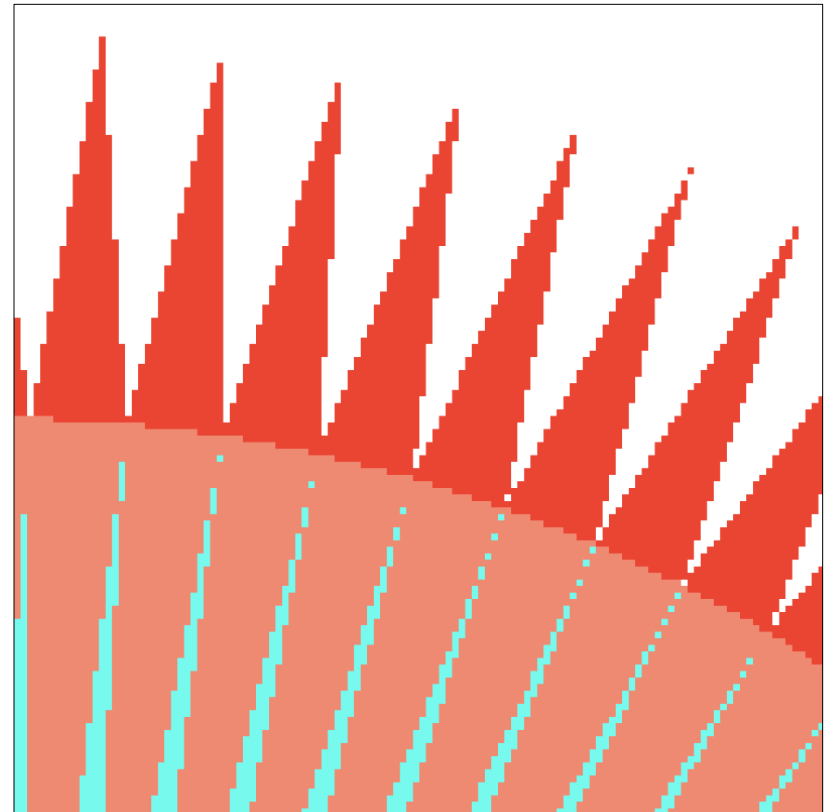
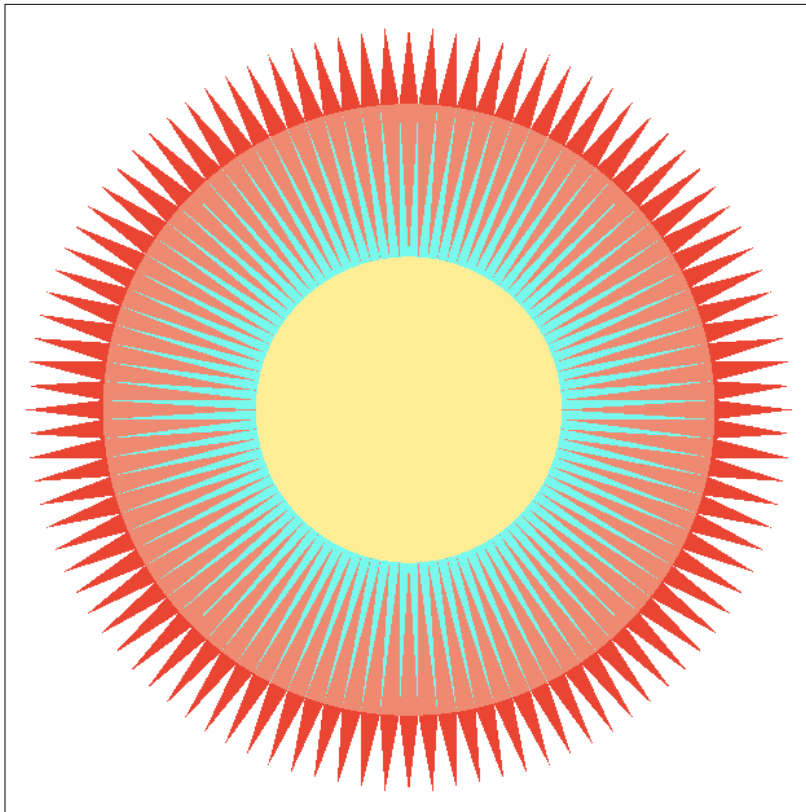
(remove frequencies above Nyquist)

Sample

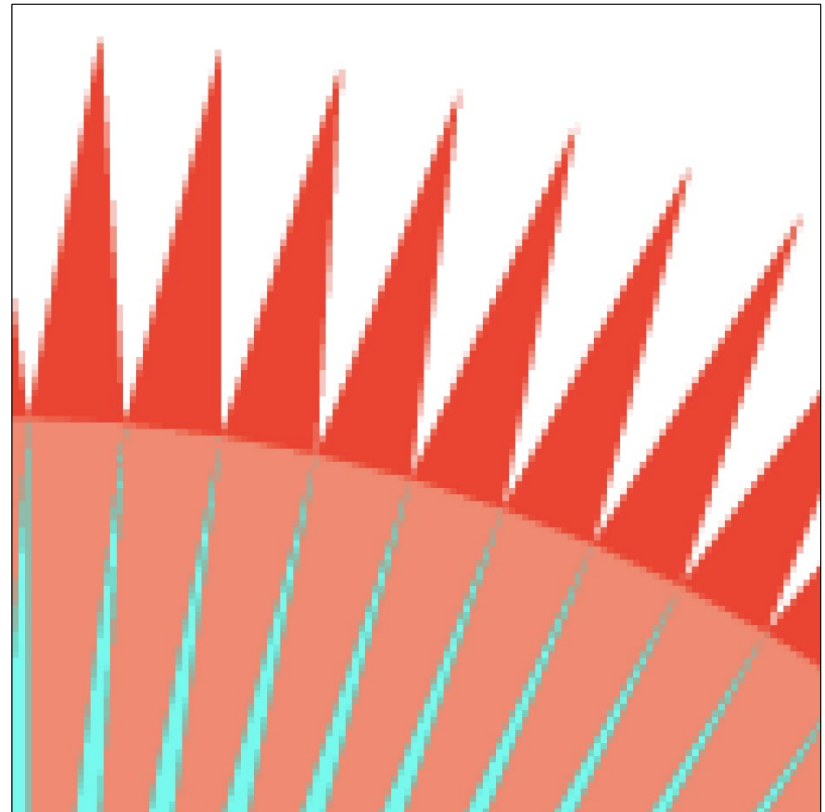
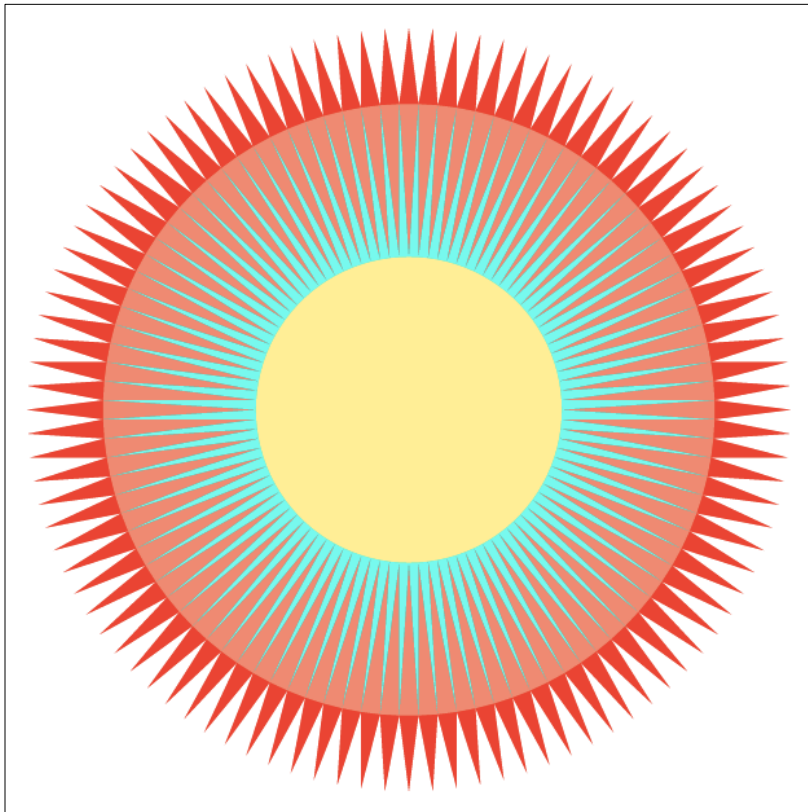
Note antialiased edges in rasterized triangle where pixel values take intermediate values



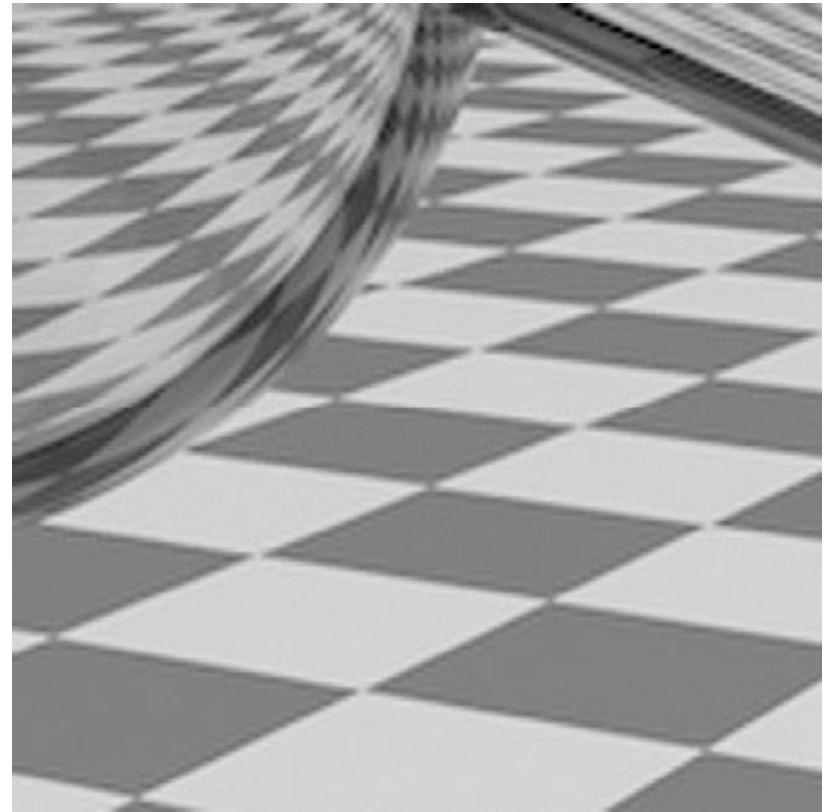
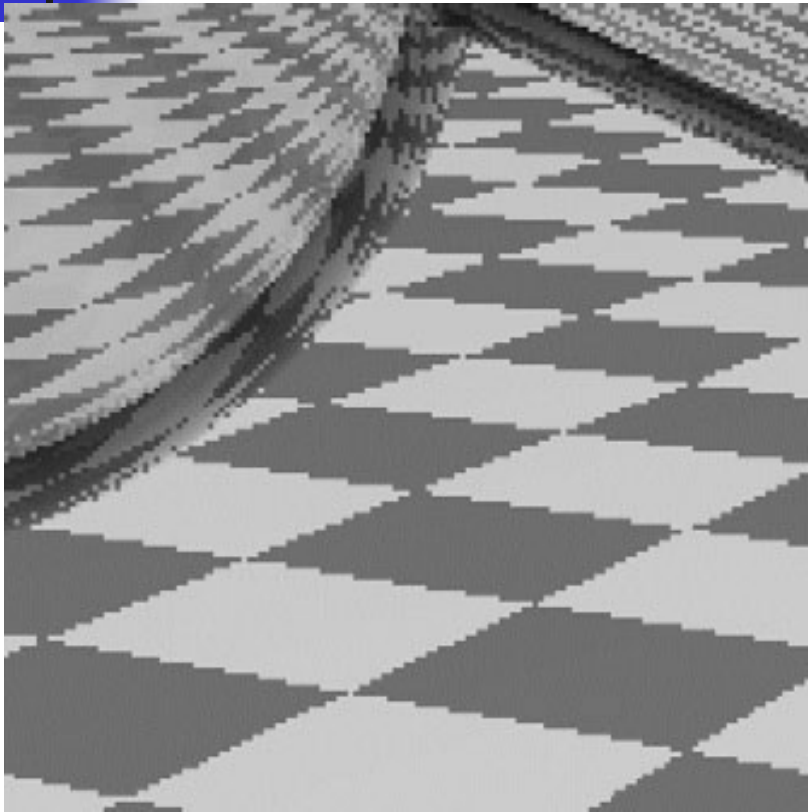
Point Sampling



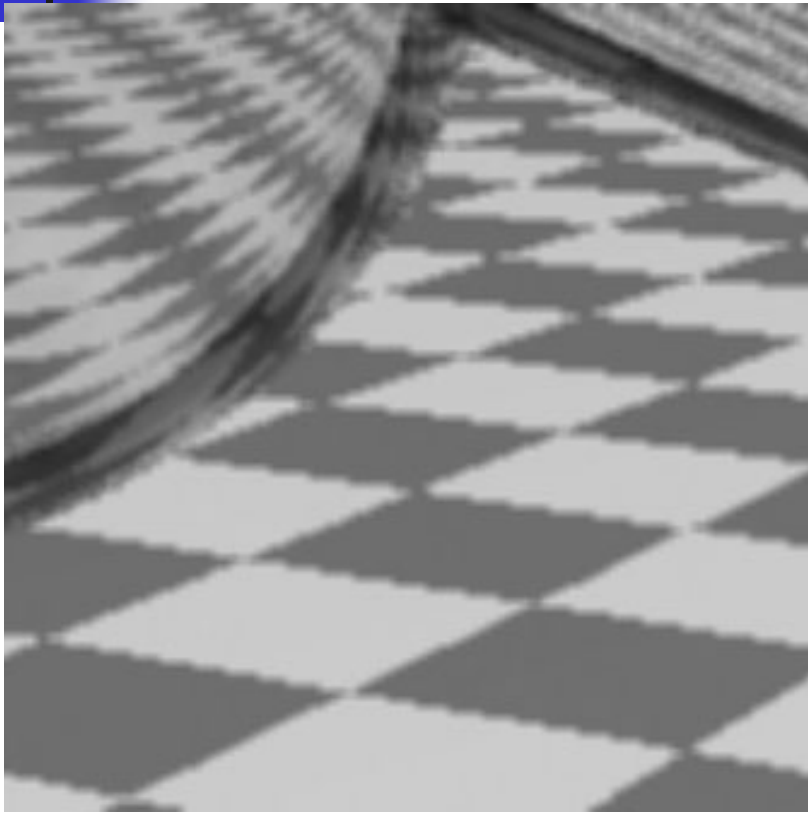
Antialiasing



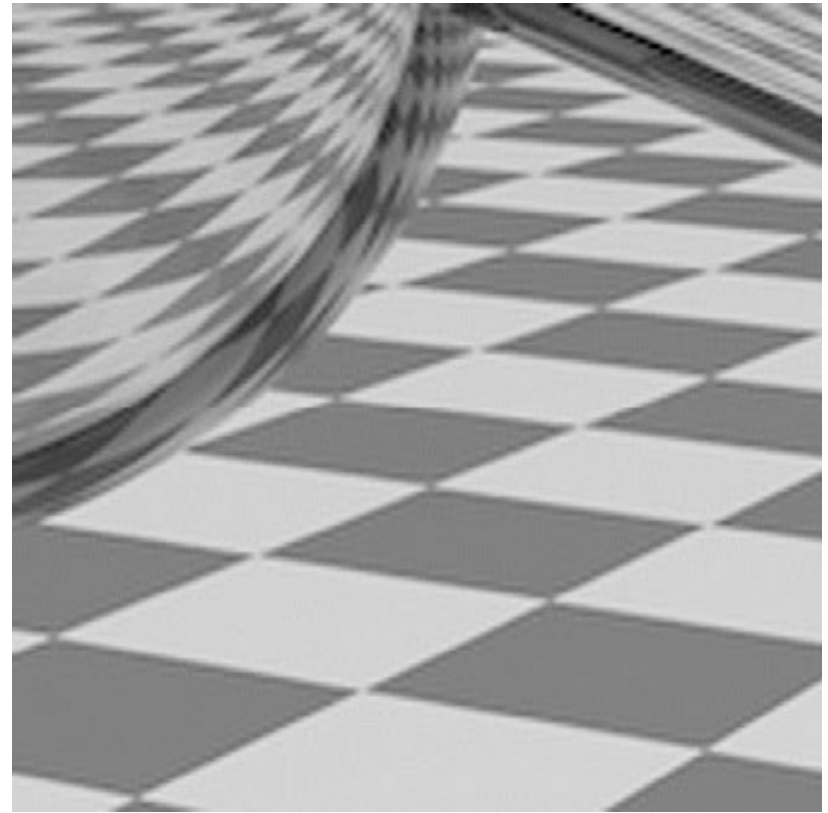
Point Sampling vs Antialiasing



Antialiasing vs Blurred Aliasing



(Sample then filter, WRONG!)

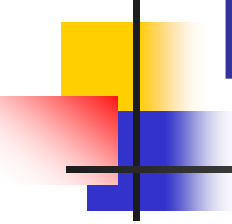


(Filter then sample)



UNIVERSITY *of* **HOUSTON**

DEPARTMENT OF COMPUTER SCIENCE

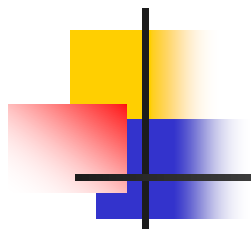


But why?

1. Why under-sampling introduces aliasing?
2. Why pre-filtering then sampling can do antialiasing?

Let's look at the reasons in the next lecture.





Thank you!

(And thank Prof. lingqi Yan, Prof. Ravi Ramamoorthi and Prof. Ren Ng for many of the slides!)



UNIVERSITY *of* **HOUSTON**

DEPARTMENT OF COMPUTER SCIENCE